QUESTION 1 [18 MARKS]

Five processes, **A, B, C, D**, and **E**, arrive at different times, with varying CPU burst lengths (in milliseconds) and priority levels, as shown in **Table 3**.

Table 3

Processes	Arrival Time	Burst Time	Priority
A	0	12	6
B	2	9	4
C	4	6	3
D	6	10	1
E	10	4	2

a) Draw Gantt chart of the scheduling if

i. First Come First Serve (FCFS)
ii. Shortest Job First (SJF)
iii. Shortest Remaining Time First (SRTF)
iv. Non-Preemptive Priority Scheduling
v. Preemptive Priority Scheduling
vi. Round Robin (q = 4)

b) Calculate completion time, turnaround time and waiting time for each process in table format for each algorithm.

c) Calculate average of turnaround time and average of waiting time for each algorithm.

QUESTION 2 [20 MARKS]

There are 3 processes in the memory space, namely processes X, Y, Z, and the dispatcher as shown in Table 1. These processes will be executed on a single processor with a single core. Process X makes an I/O resource request at the 5th instruction cycle for 25 instruction cycles. Process Y also makes a request for the I/O resource at the 39th instruction cycle and waits for the resource for 30 instruction cycles.

Table 1

Process	Addresses
X	5000-5013
Y	3000-3009
Z	1000-1017
Dispatcher	100-105

a) Show your working by using the gantt chart to trace the process states for up to 75 instruction cycles. You may assume that the three processes are run concurrently on a uniprocessor and that the maximum CPU time for each process is 6 instruction cycles.
[12 marks]

b) Which jobs has been **Unsuccessfully Executed**? [2 marks]

c) What is the **Turnaround Time** of **Process Z**? [2 marks]

d) What is the **Waiting Time** of **Process Z**? [2 marks]

e) What is **the states** of processes X, Y and Z at $\mathbf{t_{15}}$? [3 marks]

f) What is **the states** of processes X, Y and Z at $\mathbf{t_{40}}$? [3 marks]

QUESTION 3 [12 MARKS]

A circular buffer with 12 slots is used to handle print requests between a system's Text Editor (producer) and Print Manager (consumer). At T = 0, both processes share a common semaphore "BufferCount", having an initial value = 9. Complete Table 3 (for items labelled as **a** – **h**) by updating the value of BufferCount and Event description, assuming that the Text Editor adds jobs to the buffer and the Print Manager removes them for printing.

Time	Text Editor	BufferCount	Print Manager	Event
T0		9		BufferCount is initialized at 9
T1	1	**(a)**		**(e)**
T2		**(b)**	1	**(f)**
T3	1	**(c)**		**(g)**
T4		**(d)**	1	**(h)**

Table 3

QUESTION 4 [15 MARKS]

Figure 2 shows two processes, Process A and Process B, that share the same variable x initialized to x = 100. Process A wants to increment the variable x, while Process B wants to decrement the variable x. Both run in an infinite loop.

```
do {
      acquire();
      x += 1
      release();
}while(True)//process A
```

Process A

```
do {
      acquire();
      x -= 1
      release();
}while(True); //ProcessB
```

Process B

Figure 2

a) Complete the definition of the acquire() and release() function as shown in the Figure 3 below (for items labelled as **a** and **b**). [2 marks]

```
acquire() {
      while(!available);
      _____ ( a ) ____;
}
```

acquire()

```
release() {
      _____ ( b ) ____;
}
```

release()

Figure 3

b) Assume that the CPU scheduler schedules the processes alternately, i.e., Process A, Process B, Process A, Process B. Continue trace the instruction of the processes as well as the value of `x` and `available` in the Table 4 below until t=10.

[5 marks]

Copy Table 4 to your answer booklet to answer the question:

time	Process A	Process B	Available	X
T0	Acquire()	Acquire()	True	X=100
T1	X+=1	Acquire()	False	X=101
T2	Release()	Acquire()	True	X=101
…				
T10				

Table 4

c) Modify the code in Figure 2 to use semaphores using the correct semaphore operations. [3 marks]

d) Using the semaphore algorithm, trace the execution of Process A and Process B (as in Question 4 (a) and (b)) by stating the instructions executed by Process A and B as well as the value of X and the integer semaphore S. Set the initial condition of S=1 and the processes execute alternately beginning with Process A. [5 marks]

Copy Table 5 to your answer booklet to answer the question:

Time	Process A	Process B	S	X
T0	wait()	wait()	1	100
T1	x+=1	wait()	0	101
T2				
…				
T10				

Table 5

QUESTION 5 [15 MARKS]

a) Figure 4 shows the implementation of `Producer-Consumer` using `Semaphores`. Supposed the initial values of the buffer size `N=8, mutex=1, empty=6` and `full=2`.

Producer	**Consumer**

`do {`
`  produce an item`
`  …`
`  wait(empty);`
`  wait(mutex);`
`  …`
`  add item to buffer`
`  …`
`  signal(mutex);`
`  signal(full);`
`} while (TRUE);`
	`do {`
	`  …`
	`  wait(full);`
	`  wait(mutex);`
	`  …`
	`  remove an item from buffer`
	`  …`
	`  signal(mutex);`
	`  signal(empty);`
	`} while (TRUE);`

Figure 4

Complete the value of the appropriate `full`, `mutex` and `empty` variables in Table 6 (for items labelled as **a** – **l**) if the following processes sequence of order is executed:

Consumer, Consumer, Producer

[6 marks]

Process	Wait(full)	Wait(mutex)	Signal (mutex)	Signal (empty)
Initial Value	2	1		6
Consumer	**(a)**	**(b)**	**(c)**	**(d)**
Consumer	**(e)**	**(f)**	**(g)**	**(h)**
Process	Wait(empty)	Wait(mutex)	Signal (mutex)	Signal (full)
Producer	**(i)**	**(j)**	**(k)**	**(l)**

Table 6

b) There are 2 concurrent processes wishing to enter their Critical Sections. Using Paterson's algorithm, complete Table 7 below (for items labelled as **a** – **o**). [9 marks]

Time	flag[0]	flag[1]	turn	Events
t_0	FALSE	FALSE	0	P0 executes RS0
t_1	FALSE	TRUE	**(a)**	**(b)**
t_2	FALSE	TRUE	**(c)**	**(d)**
t_3	**(e)**	TRUE	1	**(f)**
t_4	TRUE	TRUE	**(g)**	**(h)**
t_5	TRUE	FALSE	**(i)**	**(j)**
t_6	TRUE	FALSE	**(k)**	**(l)**
t_7	FALSE	FALSE	**(m)**	**(n)**
t_8	FALSE	TRUE	**(o)**	P1 requests to enter CS1

Table 7